Loan Approval Prediction

An End-to-End Machine Learning Pipeline

Ana Nicuesa

July 7, 2026

Agenda

1. Business Problem
2. Dataset
3. Exploratory Data Analysis
4. Preprocessing & Feature Engineering
5. Modeling
6. Results
7. Explainability & Fairness
8. Deployment
9. Conclusions

Business Problem

- Manual loan underwriting is slow and inconsistent across reviewers.
- Goal: predict whether a loan application will be **approved** or **rejected**, using applicant and loan attributes.
- A reliable model can:
  - Speed up decisions on straightforward applications
  - Flag borderline cases for manual review
  - Provide a consistent baseline to audit human decisions against
- Approving a risky applicant and rejecting a creditworthy one have different costs $\rightarrow$ optimize for **ROC AUC**, not just accuracy.

Dataset

563 loan applications, 13 columns:

- Applicant: gender, marital status, dependents, education, self-employment
- Financial: applicant/coapplicant income, loan amount, loan term
- Credit history (meets guidelines: yes/no)
- Property area (urban / semiurban / rural)
- Target: `loan_status` (approved / rejected)

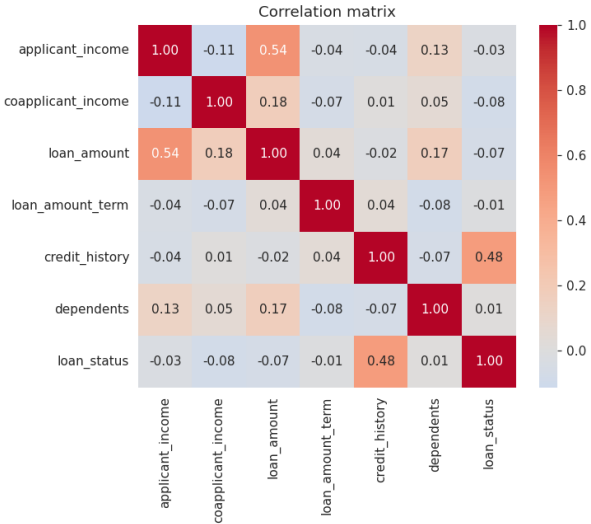
Data quality issues found:

- Missing values in *every* column (3–6%)
- Exact duplicate rows
- Repeated `loan_ids` (resubmitted applications)
- Target missing in some rows

Key Findings from EDA

- Class balance: **71% approved / 29% rejected** $\rightarrow$ moderate imbalance
- **Credit history** is the strongest signal by far:
  - Approval rate with good credit history: $\approx$80%
  - Approval rate without it: $\approx$11%
- Raw income and loan amount barely correlate with approval on their own
- Property area and marital status show smaller effects (semiurban and married applicants approved slightly more often)
- Income and loan amount are right-skewed with a few high-income outliers

Correlation with the Target



Preprocessing Pipeline

1. **Cleaning:** drop exact duplicates, drop rows with missing target, drop identifier column, normalize `dependents` ("3+" $\rightarrow$ 3)
2. **Feature engineering:**
   - `total_income` = applicant + coapplicant income
   - `loan_income_ratio` = loan amount / total income
3. **Train/test split:** 80/20, stratified on target, fixed random seed
4. **Imputation, scaling, encoding:** handled inside a `ColumnTransformer`, fit only on the training fold within cross-validation — no leakage
   - Numeric: median imputation + standard scaling
   - Categorical: most-frequent imputation + one-hot encoding

Modeling Approach

- Three candidate models, each wrapped in the same `Pipeline` (preprocessing + classifier):
  - Logistic Regression
  - Random Forest
  - XGBoost
- 5-fold **stratified cross-validation**, scoring on ROC AUC
- **GridSearchCV** for Logistic Regression (small search space)
- **RandomizedSearchCV** for Random Forest and XGBoost (larger, continuous search spaces)
- Best pipeline persisted with `joblib` for reuse in evaluation and deployment

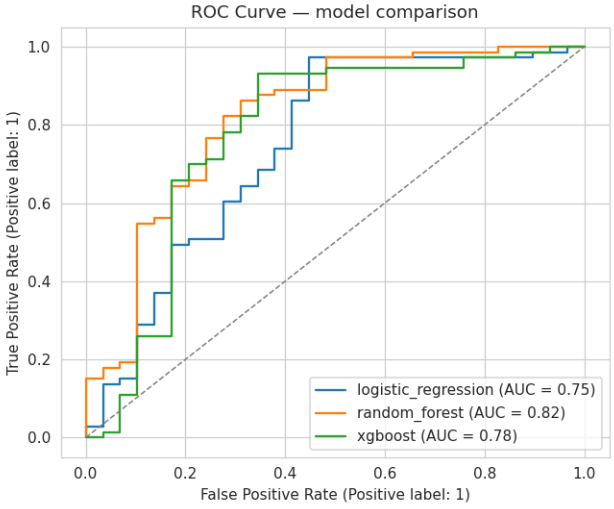
Model Comparison

Model	Accuracy	Precision	Recall	F1	ROC AUC
Random Forest	0.804	0.827	0.918	0.870	**0.818**
XGBoost	0.814	0.865	0.877	0.871	0.776
Logistic Regression	0.833	0.826	0.973	0.893	0.746

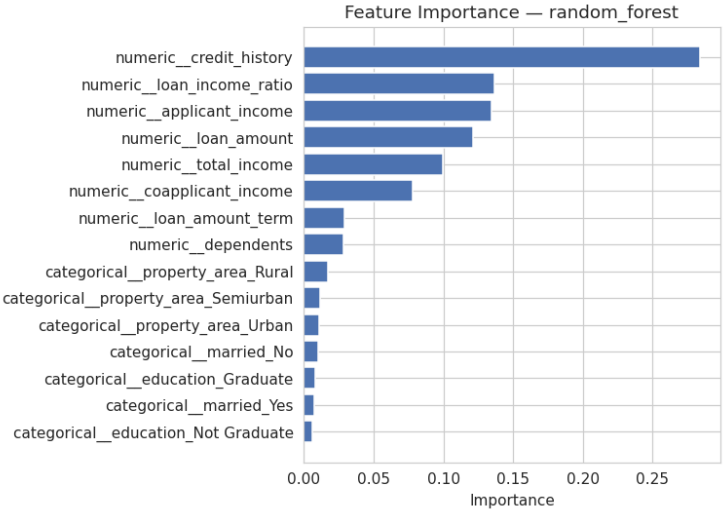
Table: Test-set performance, best model in bold

Random Forest selected as the production model based on ROC AUC.

ROC Curve Comparison



Feature Importance — Random Forest



Confirms the EDA: credit history dominates the prediction.

Model Explainability with SHAP

- Feature importance shows *what* matters; SHAP also shows *which direction* and *for whom*
- Global: mean absolute SHAP value bar chart + beeswarm plot (magnitude *and* direction, per applicant)
- Local: waterfall plot per applicant, decomposing one prediction into feature contributions that sum exactly to the model's output
- SHAP mostly agrees with the Random Forest's built-in importance ranking, but adds direction, per-instance detail, and output-scale units — impurity importance gives none of these

Global Feature Impact — SHAP Beeswarm

Blue = low feature value, red = high. Credit history flips sign completely depending on its value.

Local Explanations: Three Applicants

- **Strong applicant** (credit history OK, solid income): approved, 95% — credit history and loan size drive it
- **Weak applicant** (no credit history, high loan request): rejected, 14% — credit history alone contributes -0.46
- **Borderline applicant** (modest income, good credit history): approved, 93% — credit history outweighs a middling loan-to-income ratio

Same three profiles used in the Streamlit app's sanity checks and in `04_Model_Evaluation.ipynb` — defined once in `src/utils.py`, no duplicated data.

Important Caveat

- SHAP explains what the **model** learned from historical data — not a causal relationship in the real world
- A large SHAP value means the model relies on that feature; it does *not* mean changing it would change a real applicant's outcome
- Correlated features (`applicant_income`, `coapplicant_income`, `total_income`) can split credit for the same signal in ways that look inconsistent at a glance
- Training labels reflect *past* approval decisions, which may themselves encode bias that SHAP will faithfully explain rather than correct

Fairness Check

- `gender`, `married`, and `dependents` are present in the data and are sensitive attributes in real lending (ECOA in the US prohibits credit discrimination on sex and marital status)
- `gender` has the **lowest** mean absolute SHAP value of any feature in the model
- `married` and `dependents` show a small but non-zero effect — well below `credit_history` or the income features
- Risks that remain regardless: **proxy discrimination** (income correlates with marital status) and **historical bias** baked into the labels
- Conclusion: low SHAP magnitude is a useful signal, not a substitute for a legal/compliance fairness audit before real deployment

Streamlit Application

- Interactive web app for single-applicant predictions
- Inputs: gender, marital status, dependents, education, employment, income, loan amount/term, credit history, property area
- Output: **Approved / Rejected** with probability and confidence score
- Each prediction ships with a SHAP explanation: factors that increased vs. decreased approval probability, plus the full waterfall breakdown
- Same `predict.py` / `explain.py` logic used by the app and the notebooks — no duplicated code
- Runs locally (`streamlit run app/app.py`) or from Google Colab

Conclusions & Future Work

Conclusions

- Credit history is the single most predictive feature across every model
- Engineered affordability ratios add signal that raw income/loan figures don't provide
- Random Forest offers the best trade-off between recall and overall discrimination (ROC AUC)

Future improvements

- Formal fairness audit (demographic parity, equal opportunity) beyond SHAP magnitude alone
- Larger hyperparameter search budget with more compute
- Larger, more recent dataset to validate generalization
- Data and SHAP-attribution drift monitoring if connected to a live pipeline

Thank you

Questions?